Data Engineering Project: E-commerce Analytics Pipeline

📋 Tổng quan dự án

Dự án xây dựng pipeline xử lý và phân tích dữ liệu toàn diện cho hệ thống bán lẻ công nghệ TechStore Vietnam, bao gồm nhiều kênh bán hàng (online/offline) và cổng thanh toán.

Mục tiêu

- Tích hợp dữ liệu từ nhiều nguồn khác nhau
- Phân tích customer journey xuyên suốt các kênh
- Theo dõi cashflow và tình trạng thanh toán
- Xây dựng dashboard trực quan cho business insights

🏗️ Kiến trúc hệ thống

```
┌─────────────────────┐
│ Source APIs         │
│ (FastAPI)           │
└─────────────────────┘
        │ Generate .json.gz files
        ▼
┌─────────────────────┐
│ GCS Buckets         │
│ Raw Data            │
└─────────────────────┘
        │ Python ETL
        ▼
┌─────────────────────┐
│ BigQuery            │
│ Data Warehouse      │
└─────────────────────┘
        │ Power BI Connector
        ▼
┌─────────────────────┐
│ Power BI            │
│ Dashboards          │
└─────────────────────┘
```

📊 Nguồn dữ liệu và cấu trúc

1. E-commerce Platforms

1.1 Shopify (Online Store)

Location: `gs://minpy/shopify/`

Files:

- `products.json.gz` (1,000 products)
- `orders_batch_0.json.gz`, `orders_batch_1.json.gz`, ... (200,000 orders)
- `staff.json.gz` (300 staff members)
- `customers_batch_0.json.gz`, ... (2,000,000 customers)

Key Fields:

```json
```

```
// Products
{
  "id": int,
  "name": string,
  "sku": string,
  "category": string,
  "brand": string,
  "price_vnd": int,
  "price_usd": float,
  "barcode": string,
  "stock_quantity": int
}

// Orders
{
  "id": int,
  "order_number": string,
  "transaction_id": string,
  "customer_id": int,
  "order_date": datetime,
  "payment_gateway": string,
  "payment_status": string,
  "total_vnd": int,
  "total_usd": float,
  "line_items": array,
  "source": "shopify"
}

// Customers
{
  "id": int,
  "email": string,
  "name": string,
  "phone": string,
  "city": string,
  "country": string,
  "created_at": datetime
}
```

1.2 Sapo POS (Offline Stores)

Location: `gs://minpy/sapo/`

Files:

- `locations.json.gz` (50 stores)

- `orders_batch_0.json.gz`, ... (1,000,000 orders)

Key Fields:

```
// Orders
{
  "id": int,
  "code": string,
  "transaction_id": string,
  "location_id": int,
  "location_name": string,
  "customer": object,
  "staff": object,
  "line_items": array,
  "total_vnd": int,
  "payment_method": string,
  "status": string,
  "source": "sapo_pos"
}

// Locations
{
  "id": int,
  "code": string,
  "name": string,
  "address": string,
  "city": string,
  "phone": string
}
```

1.3 Online Orders (Multi-channel)

Location: `gs://minpy/online_orders/`

Files:

- `online_orders.json.gz` (50,000 orders)

Key Fields:

```
```

```
{
  "order_id": string,
  "transaction_id": string,
  "channel": string, // shopify, website, mobile_app, lazada, shopee
  "customer_id": int,
  "status": string,
  "line_items": array,
  "total": int,
  "payment_method": string,
  "shipping_address": object,
  "tracking": object,
  "created_at": datetime
}
```

2. Payment Gateways

2.1 PayPal

Location: `gs://minpy/paypal/`

Files:

- `transactions.json.gz` (300 transactions)

Key Fields:

```json
{
  "transaction_id": string,
  "paypal_transaction_id": string,
  "customer_id": int,
  "transaction_amount": {
    "currency_code": "USD",
    "value": string
  },
  "transaction_amount_vnd": int,
  "transaction_status": string,
  "payer_info": object,
  "transaction_initiation_date": datetime,
  "source": "paypal"
}
```

2.2 Mercury Bank

Location: `gs://minpy/mercury/`

Files:

- `accounts.json.gz` (3 accounts)
- `transactions.json.gz` (500 transactions)

Key Fields:

```json
// Transactions
{
  "id": string,
  "transaction_id": string,
  "accountId": string,
  "amount": float,
  "amount_vnd": int,
  "kind": string, // debitCardPurchase, incomingWire, etc.
  "status": string,
  "createdAt": datetime,
  "details": object,
  "source": "mercury_bank"
}
```

2.3 MoMo

Location: `gs://minpy/momo/`

Files:

- `transactions.json.gz` (500 transactions)

Key Fields:

```json
```

```
{
  "transaction_id": string,
  "orderId": string,
  "transId": int,
  "amount": int,
  "resultCode": int, // 0 = success
  "message": string,
  "payType": string,
  "responseTime": int,
  "responseTimeISO": datetime,
  "extraData": object,
  "source": "momo"
}
```

2.4 ZaloPay

Location: `gs://minpy/zalopay/`

Files:

- `transactions.json.gz` (500 transactions)

Key Fields:

```json
{
  "transaction_id": string,
  "app_trans_id": string,
  "zp_trans_id": int,
  "amount": int,
  "return_code": int, // 1 = success
  "channel": int,
  "bank_code": string,
  "server_time_iso": datetime,
  "refund_history": array,
  "source": "zalopay"
}
```

3. Tracking & Analytics

3.1 Cart Tracking

Location: `gs://minpy/cart_tracking/`

Files:

- `cart_events.json.gz` (10,000+ events)

Key Fields:

```json
{
  "event_id": string,
  "event_type": string, // add_to_cart, remove_from_cart, view_item, purchase, etc.
  "session_id": string,
  "customer_id": int, // null for guest users
  "product_id": int,
  "timestamp": datetime,
  "source": string,
  "device": string,
  "browser": string,
  "utm_source": string,
  "utm_campaign": string
}
```

🗄️ BigQuery Data Model

Schema Organization

```
project_id: your-project
dataset: techstore_analytics
```

Dimension Tables

Học viên cần tạo các bảng dimension sau trong BigQuery:

`dim_customers`

- **Mục đích**: Thông tin khách hàng
- **Partition**: Theo ngày tạo (created_at)
- **Columns**: customer_id, email, full_name, phone, city, country, created_at, customer_segment, lifetime_value_vnd, total_orders, first_order_date, last_order_date

`dim_products`

- **Mục đích**: Danh mục sản phẩm

- **Columns**: product_id, product_name, sku, barcode, category, brand, price_vnd, price_usd, stock_quantity, is_active

dim_locations

- **Mục đích**: Thông tin cửa hàng/kho
- **Columns**: location_id, location_code, location_name, location_type, city, address, phone, is_active

dim_staff

- **Mục đích**: Thông tin nhân viên
- **Columns**: staff_id, staff_code, full_name, position, email, phone, location_id, hire_date, is_active

dim_date

- **Mục đích**: Date dimension cho time-series analysis
- **Columns**: date_key, year, quarter, month, month_name, week, day_of_month, day_of_week, day_name, is_weekend, is_holiday, fiscal_year, fiscal_quarter

Fact Tables

Học viên cần tạo các bảng fact sau trong BigQuery:

fact_orders

- **Mục đích**: Tất cả đơn hàng từ mọi kênh
- **Partition**: Theo order_date_key
- **Cluster**: customer_id, channel
- **Key Columns**: order_key (primary), order_id, transaction_id, customer_id, order_date, channel, source, status, payment_status, total_vnd, total_usd

fact_order_items

- **Mục đích**: Chi tiết sản phẩm trong đơn hàng
- **Partition**: Theo order_date_key
- **Cluster**: product_id
- **Key Columns**: order_item_key (primary), order_key, transaction_id, product_id, quantity, unit_price_vnd, line_total_vnd

fact_payments

- **Mục đích**: Giao dịch thanh toán từ tất cả payment gateways
- **Partition**: Theo payment_date_key
- **Cluster**: customer_id, payment_gateway
- **Key Columns**: payment_key (primary), transaction_id, order_id, customer_id, payment_gateway, payment_method, amount_vnd, payment_status, payment_date

fact_cart_events

- **Mục đích**: Hành vi người dùng và cart tracking
- **Partition**: Theo event_date_key
- **Cluster**: customer_id, session_id, event_type
- **Key Columns**: event_key (primary), event_id, session_id, customer_id, event_type, event_timestamp, product_id, source, device, utm_source, utm_campaign

fact_bank_transactions

- **Mục đích**: Giao dịch ngân hàng Mercury
- **Partition**: Theo transaction_date_key
- **Key Columns**: transaction_key (primary), transaction_id, account_id, transaction_type, amount_vnd, status, transaction_date

Views for Analysis

Học viên cần tạo các views sau để hỗ trợ phân tích:

vw_customer_journey

- **Mục đích**: Phân tích customer journey từ touchpoint đầu tiên đến purchase
- **Yêu cầu**: Kết hợp dữ liệu từ fact_cart_events và fact_orders, tạo touchpoint_sequence cho mỗi customer

vw_cashflow_daily

- **Mục đích**: Báo cáo cashflow theo ngày
- **Yêu cầu**: Tính toán sales revenue, payment received, bank inflow/outflow, và net cashflow

vw_payment_status

- **Mục đích**: Phân loại tình trạng thanh toán của đơn hàng

- **Yêu cầu**: Join fact_orders và fact_payments, tính payment_delay_hours, outstanding_amount, phân loại payment_status_category (Paid, Pending, Overdue, etc.)

🛠️ Yêu cầu kỹ thuật

1. Python ETL Pipeline

Thư viện cần thiết:

```python
google-cloud-storage==2.10.0
google-cloud-bigquery==3.11.0
pandas==2.0.3
pyarrow==12.0.1
```

Cấu trúc project:

```
etl_pipeline/
├── config/
│   ├── gcs_config.yaml
│   └── bigquery_schema.yaml
├── extractors/
│   ├── __init__.py
│   ├── base_extractor.py
│   ├── shopify_extractor.py
│   ├── sapo_extractor.py
│   ├── payment_extractor.py
│   └── tracking_extractor.py
├── transformers/
│   ├── __init__.py
│   ├── base_transformer.py
│   ├── dimension_transformer.py
│   └── fact_transformer.py
├── loaders/
│   ├── __init__.py
│   └── bigquery_loader.py
├── utils/
│   ├── __init__.py
│   ├── gcs_helper.py
│   └── logger.py
├── orchestration/
│   ├── __init__.py
│   └── pipeline_orchestrator.py
├── tests/
│   └── test_pipeline.py
├── main.py
└── requirements.txt
```

Các bước thực hiện:

Bước 1: Extract từ GCS

- Tạo BaseExtractor class để đọc file .json.gz từ GCS bucket
- Implement methods: extract_json_gz(), list_files()
- Xử lý decompress và parse JSON data

Bước 2: Transform Data

- Tạo DimensionTransformer cho dimension tables
- Tạo FactTransformer cho fact tables
- Standardize column names, data types

- Generate unique keys (order_key, payment_key, etc.)
- Handle missing values và data quality issues

Bước 3: Load to BigQuery

- Tạo BigQueryLoader class
- Implement methods: create_dataset_if_not_exists(), load_dataframe(), execute_query()
- Xử lý write_disposition (WRITE_TRUNCATE, WRITE_APPEND)
- Error handling và logging

Bước 4: Orchestration

- Tạo PipelineOrchestrator class để điều phối toàn bộ pipeline
- Load dimensions trước, sau đó load facts
- Update aggregates (lifetime_value, total_orders, etc.)
- Comprehensive logging và error handling

2. Data Quality Checks

Học viên cần implement các data quality checks sau:

Null Value Checks

- Kiểm tra null values trong các columns quan trọng (customer_id, order_id, transaction_id, amounts)
- Log warning nếu phát hiện null values

Duplicate Checks

- Kiểm tra duplicate records dựa trên key columns
- Identify và remove/flag duplicates

Date Range Validation

- Verify dates nằm trong khoảng hợp lý
- Warning nếu có future dates
- Check min/max date range

Amount Validation

- Check for negative amounts (trừ bank transactions)

- Identify outliers hoặc unreasonable amounts
- Statistical summary (mean, median, std dev)

📊 Power BI Dashboard Requirements

Dashboard 1: Customer Journey Analytics

Mục tiêu: Phân tích hành trình khách hàng từ lần đầu tiếp xúc đến mua hàng

Visuals cần có:

1. **Sankey Diagram - Customer Journey Flow**
    - Trục X: Touchpoint sequence (1st touch → 2nd touch → ... → Purchase)
    - Flow: Từ channel này sang channel khác
    - Metrics: % customers at each stage
2. **Funnel Chart - Conversion Funnel**
    - View Product → Add to Cart → Begin Checkout → Purchase
    - Conversion rate tại mỗi stage
    - Filter by: Date range, UTM source, Device
3. **Time to Purchase Analysis**
    - Line chart: Average days from first touch to purchase
    - Grouped by: Customer segment, Channel
    - Trend over time
4. **Channel Attribution**
    - Stacked bar chart: First touch vs Last touch attribution
    - Revenue contribution by channel
    - Multi-touch attribution model
5. **Cart Abandonment Analysis**
    - Abandonment rate by stage
    - Top abandoned products
    - Recovery rate
    - Time-based pattern (hour of day, day of week)
6. **Customer Segmentation Matrix**
    - Scatter plot: Recency vs Frequency vs Monetary

- Color-coded by segment (VIP, Regular, At-risk, New)
- Bubble size = Lifetime value

DAX Measures cần tạo:

- Conversion Rate: Tính tỷ lệ từ view_item đến purchase
- Average Time to Purchase: Số ngày trung bình từ first touch đến purchase
- Cart Abandonment Rate: Tỷ lệ add_to_cart không chuyển thành purchase
- Customer Lifetime Value: Tổng giá trị đơn hàng của mỗi customer
- Session Conversion Rate: % sessions có purchase

Dashboard 2: Cashflow & Financial Analytics

Mục tiêu: Theo dõi dòng tiền vào/ra, dự báo cashflow

Visuals cần có:

1. **Waterfall Chart - Daily Cashflow**
    - Starting balance
    - - Sales revenue
    - - Payment received
    - - Payment fees
    - - Operating expenses (from bank transactions)
    - = Ending balance
    - Time period selector: Daily/Weekly/Monthly
2. **Line Chart - Cashflow Trend**
    - 3 lines: Revenue, Payments Received, Net Cashflow
    - Moving average (7-day, 30-day)
    - Forecast for next 30 days
3. **Donut Charts - Revenue Breakdown**
    - By Channel (Shopify, Sapo, Online marketplaces)
    - By Payment Method
    - By Customer Segment
4. **KPI Cards**

- Total Revenue (MTD, YTD)
- Outstanding Receivables
- Average Collection Period (days)
- Cash Conversion Cycle

5. **Payment Gateway Performance**
    - Table: Gateway | Transactions | Volume | Fees | Success Rate
    - Sorted by volume
    - Conditional formatting for success rate
6. **Bank Balance Tracking**
    - Line chart: Account balance over time (for each bank account)
    - Alerts for low balance
    - Reconciliation status

DAX Measures cần tạo:

- Total Revenue: Tổng doanh thu từ orders completed/delivered
- Payment Collection Rate: % payment đã thu trên tổng revenue
- Outstanding Receivables: Tổng số tiền chưa thu từ orders pending
- Average Collection Period: Số ngày trung bình từ order đến payment
- Cash Balance: Số dư tài khoản ngân hàng hiện tại
- Net Cashflow: Inflow - Outflow

Dashboard 3: Payment Status & Risk Management

Mục tiêu: Giám sát tình trạng thanh toán, phát hiện rủi ro

Visuals cần có:

1. **Payment Status Overview**
    - Donut chart: Paid / Pending / Overdue / Failed
    - Value = Total amount
    - Drill-through to detail table
2. **Aging Analysis**
    - Stacked column chart: Days overdue (0-7, 8-14, 15-30, 30+)
    - Y-axis: Outstanding amount
    - Color gradient (green to red)

3. **Payment Failure Analysis**
    - Table: Gateway | Failure Rate | Top Failure Reasons
    - Trend: Failure rate over time
    - Filter by amount range
4. **High-Risk Customers**
    - Table: Customer | Outstanding Amount | Days Overdue | Order Count
    - Sorted by risk score
    - Red flag indicator for > 30 days overdue
5. **Payment Method Performance**
    - Matrix: Payment Method × Success Rate × Avg Processing Time
    - Heatmap visualization
    - Compare across channels
6. **Geographic Payment Insights**
    - Map: Payment success rate by city/province
    - Bubble size = Transaction volume
    - Color = Success rate

DAX Measures cần tạo:

- Payment Status: Phân loại Paid / Pending / Overdue / Failed
- Days Overdue: Số ngày quá hạn cho pending orders
- Payment Failure Rate: % payment failed trên tổng transactions
- Risk Score: Điểm đánh giá rủi ro dựa trên days overdue và outstanding amount
- Gateway Success Rate: % successful payments theo từng gateway

Dashboard Design Guidelines

Filters & Slicers:

- Date range picker (Last 7/30/90 days, YTD, Custom)
- Channel selector (Multi-select)
- Customer segment
- Product category
- Payment gateway

Interactivity:

- Cross-filtering between visuals
- Drill-through to detail pages
- Tooltips with additional metrics
- Bookmarks for common views

Refresh Schedule:

- Daily refresh at 6:00 AM
- Real-time dashboard for critical metrics (optional)

✅ Deliverables

1. Python ETL Code (40 points)

- [ ] Complete extractor classes for all data sources
- [ ] Transformer classes with data quality checks
- [ ] BigQuery loader with error handling
- [ ] Orchestration script with logging
- [ ] Unit tests (coverage > 80%)
- [ ] Requirements.txt with all dependencies
- [ ] README with setup instructions

2. BigQuery Schema (20 points)

- [ ] All dimension tables created with proper partitioning
- [ ] All fact tables created with clustering
- [ ] Views for analysis
- [ ] Documentation of each table/column
- [ ] Sample queries for common analytics

3. Power BI Dashboards (30 points)

- [ ] Customer Journey Dashboard (interactive)
- [ ] Cashflow Dashboard (real-time metrics)
- [ ] Payment Status Dashboard (alerts enabled)
- [ ] All DAX measures documented
- [ ] Published to Power BI Service

4. Documentation (10 points)

- [ ] Architecture diagram
- [ ] Data dictionary
- [ ] ETL process flow
- [ ] Dashboard user guide
- [ ] Troubleshooting guide

🎯 Evaluation Criteria

Criteria	Weight	Requirements
Code Quality	25%	Clean, documented, follows PEP 8, reusable
Data Accuracy	25%	Correct transformations, data integrity checks
BigQuery Design	20%	Proper schema, optimized queries, partitioning
Dashboard Quality	20%	Insightful, interactive, well-designed
Documentation	10%	Complete, clear, professional

💡 Bonus Challenges (Optional)

1. **Incremental Loading**: Implement CDC (Change Data Capture) để chỉ load dữ liệu mới/thay đổi
2. **Data Lineage**: Tracking nguồn gốc dữ liệu và transformations
3. **Alerting**: Email/Slack alerts khi phát hiện anomalies
4. **ML Integration**: Customer churn prediction model
5. **Real-time Dashboard**: Streaming data với Pub/Sub + Dataflow

---

📚 Resources

GCS & BigQuery

- BigQuery Best Practices
- Partitioning & Clustering

Power BI

- Power BI Documentation
- DAX Functions Reference

Python

- pandas Documentation
- google-cloud-python

---

5. **Connect Power BI**:

- Use DirectQuery to BigQuery
- Import data model
- Build dashboards

📞 Support

Nếu gặp vấn đề, liên hệ:

- Email: minh2210.nina@gmail.com
- Phone: 0911035269

Good luck! 🎉